

3 Measuring LLM Branching Factor

Probability Concentration and the Branching Factor. The generative process of language models can be viewed as moving down a branching tree, with each token choice selecting a path forward. While the theoretical search space spans $O(|V|^N)$ sequences for vocabulary size $|V|$ and fixed sequence length N , LLMs concentrate the vast majority of probability mass on a much smaller subset of trajectories (Holtzman et al., 2020; Hewitt et al., 2022). This high-probability subset forms a complex, sparse “effective tree” $\mathcal{T}$.

To quantify the size of this effective tree, we utilize the concept of exponentiated entropy (perplexity). We define the effective set size $|\mathcal{T}|$ as:

$$|\mathcal{T}| \stackrel{\text{def}}{=} \exp(\tilde{H}(Y_{1:N}|x;\theta)). \quad (5)$$

Information-theoretically, $|\mathcal{T}|$ reflects the size of a uniform distribution (a “fair die”) that would possess the same total uncertainty (entropy) as the model’s actual complex distribution over sequences of length N (O’Connor, 2013).

Defining Branching Factor via Balanced Tree Model. Because the exact topology of the effective tree $\mathcal{T}$ is irregular and intractable, we map it to an *equivalent balanced B -ary tree* of the same depth N . A perfectly balanced tree with constant branching factor B and depth N contains B^N leaf nodes. By equating this theoretical leaf count to the effective set size ($B^N = |\mathcal{T}|$), we derive the **Branching Factor (BF)** as the geometric mean of the branching width:

$$B \equiv B(x;\theta) \stackrel{\text{def}}{=} |\mathcal{T}|^{1/N} = \exp\left(\frac{1}{N}\tilde{H}(Y_{1:N}|x;\theta)\right) = \exp(\bar{H}(Y_{1:N}|x;\theta)) \quad (6)$$

where $\bar{H}(Y_{1:N}|x;\theta)$ denotes the length-averaged marginal entropy of the sequence. $B(x;\theta)$ thus provides a normalized, token-invariant metric: it quantifies the effective number of plausible next-token choices available to the model on average at any given step.

Linking Entropy and Log-Likelihood in Long Sequences. Calculating the exact Branching Factor requires the total entropy $\tilde{H}(Y_{1:N}|x;\theta)$. As discussed in § 2, computing this quantity directly is intractable due to the exponential number of possible trajectories. A standard Monte Carlo approach uses the *realized entropy* $h_{\text{realized}}(y_{1:N})$ of sampled sequences as a proxy. Since h_{realized} is an unbiased estimator (Eq. 4), averaging it over sufficient samples converges to the true total entropy.

However, for long sequences, even calculating h_{realized} becomes computationally prohibitive. It requires a full summation over the vocabulary V at every generation step to compute the local entropy, incurring a total cost of $O(N \cdot |V|)$. In contrast, computing the sequence’s negative log-likelihood (NLL) involves only the probabilities of the selected tokens, scaling linearly as $O(N)$. To enable efficient estimation for long horizons, we effectively need a second level of approximation: using the computationally cheap NLL as a proxy for the realized entropy.

Standard Asymptotic Equipartition Property (AEP) theory (Shannon, 1948) suggests that for stationary processes, the length-averaged NLL of a typical sequence will converge to the (length-averaged) total entropy. However, LLM generation is neither stationary nor ergodic. Fortunately, Mudireddy et al. (2024) demonstrate that a robust connection persists without these strict assumptions: the NLL converges to the *realized entropy* h_{realized} instead. We characterize this relationship as follows:⁵

Theorem 3.1 (Log-Likelihood Convergence for LLMs) *Given $0 < \epsilon < 1$, as $N \rightarrow \infty$:*

$$P\left(\left|-\frac{1}{N}\log \tilde{P}(y_{1:N}|x;\theta) - \frac{1}{N}h_{\text{realized}}(y_{1:N})\right| < \epsilon\right) \rightarrow 1 \quad (7)$$

Since $\mathbb{E}[h_{\text{realized}}] = \tilde{H}(Y_{1:N}|x;\theta)$, this theorem justifies using the NLL of sampled sequences as a *proxy* for realized entropy in long horizons, provided the variance is low. As an empirical verification for Theorem 3.1, we plot NLL and realized entropy for sampled outputs of Llama-3-8B-Instruct over multiple datasets⁶ in

⁵We provide a simplified proof in § H with minor changes to the original proof of Mudireddy et al. (2024). Notably, our goal is only to show the approximation between length-averaged log-likelihood and entropy for a typical sequence, so we do not require stricter assumptions like ergodicity or stationarity.

⁶For dataset-specific details, we refer readers to § B.

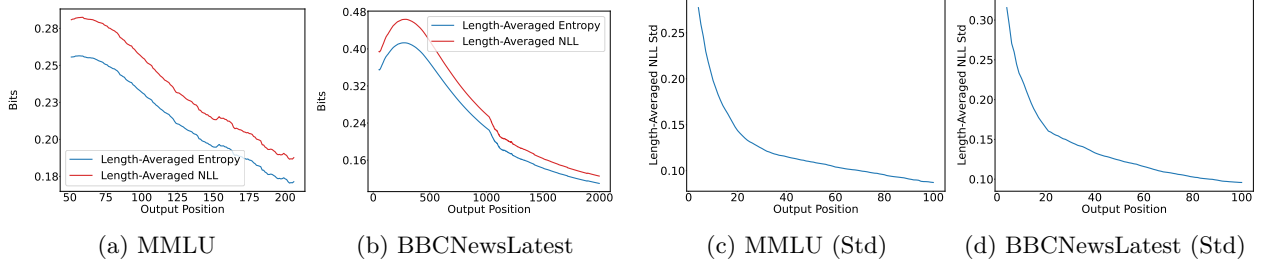


Figure 2: **Convergence of NLL and Entropy.** (a, b): The length-averaged NLL closely tracks the length-averaged Entropy. (c, d): The standard deviation of the length-averaged NLL diminishes rapidly with output length.

Figure 2. We observe that: as output length increases, the deviation between NLL and realized entropy vanishes, and the standard deviation of the estimator diminishes rapidly (within the first 10 tokens).

Empirical Estimator for Branching Factor. We now translate the theoretical definition of $B(x; \theta)$ into a practical estimator $\tilde{B}(x; \theta)$. This requires addressing two practical realities: (1) we rely on finite Monte-Carlo samples rather than full distribution access, and (2) while our theory assumes a fixed N , generation in practice terminates dynamically upon emitting an EOS token.

We estimate BF using M independent sampled sequences $y^{(1)}, \dots, y^{(M)}$. To handle the computational constraints described above, we adopt a *hybrid estimator*. For short sequences, we compute the exact realized entropy $\tilde{H}(Y_{1:|y|}|x; \theta)$ at every step (since the vocabulary is truncated to V_t , this is tractable). To avoid the GPU memory bottleneck of computing the full distribution for long sequences, we approximate entropy using NLL, a substitution justified by Theorem 3.1.

We define the estimator $\tilde{B}(x; \theta)$ by averaging over the sampled trajectories:

$$\tilde{B}(x; \theta) \approx \exp \left(\frac{1}{M} \sum_{i=1}^M \mathcal{E}(y^{(i)}) \right), \quad \mathcal{E}(y) = \begin{cases} \frac{1}{|y|} h_{\text{realized}}(Y_{1:|y|}|x; \theta) & \text{if } |y| < L_\tau \\ -\frac{1}{|y|} \log \tilde{P}(y|x; \theta) & \text{otherwise} \end{cases} \quad (8)$$

where $|y|$ is the realized length of the sample (up to EOS) and L_τ is a threshold length. Note that explicitly normalizing by the realized length $|y|$ adapts our fixed- N theory to variable-length practice, effectively measuring the branching rate per *active* generation step.

Finally, to obtain the task-level Branching Factor, we average over the dataset X : $\tilde{B}(X; \theta) = \sum_x p(x) \tilde{B}(x; \theta)$. Unless otherwise specified, BF refers to this dataset-level branching factor in the following sections.

4 Benchmarking and Attributing Branch Factors

Models and Sampling. We run experiments on models from Llama-2 (Touvron et al., 2023) and Llama-3 (Dubey et al., 2024) families as they are widely-used open-weight model families. For each model family, we include both base and aligned models to investigate how alignment tuning affects BF. We set $p=0.9$ and $T=1.0$ to sample outputs to conform with the setting for most datasets.

We set $M=50$ sequences to estimate BF, which yields a reliable estimation across datasets in prior studies. For aligned models, we apply the official chat templates to prompts. In addition, we carefully control the lengths of all inputs plus outputs to be within the context window of the models.

Tasks. We consider a variety of tasks covering common application scenarios of LLM generation, including reasoning and open-ended generation: MMLU (Hendrycks et al., 2021) (Reasoning), COGNAC (Chen et al., 2022) (Controlled Generation), BBCLATESTNEWS (Li et al., 2024b) (News Generation), and CREATIVE STORYGEN (Chakrabarty et al., 2024) (Creative Generation). To test subjective randomness bias (Bigelow et al., 2024), we also prepare a synthetic task RANDOM STRINGS where the prompt is generated via random characters. See § B for dataset details.

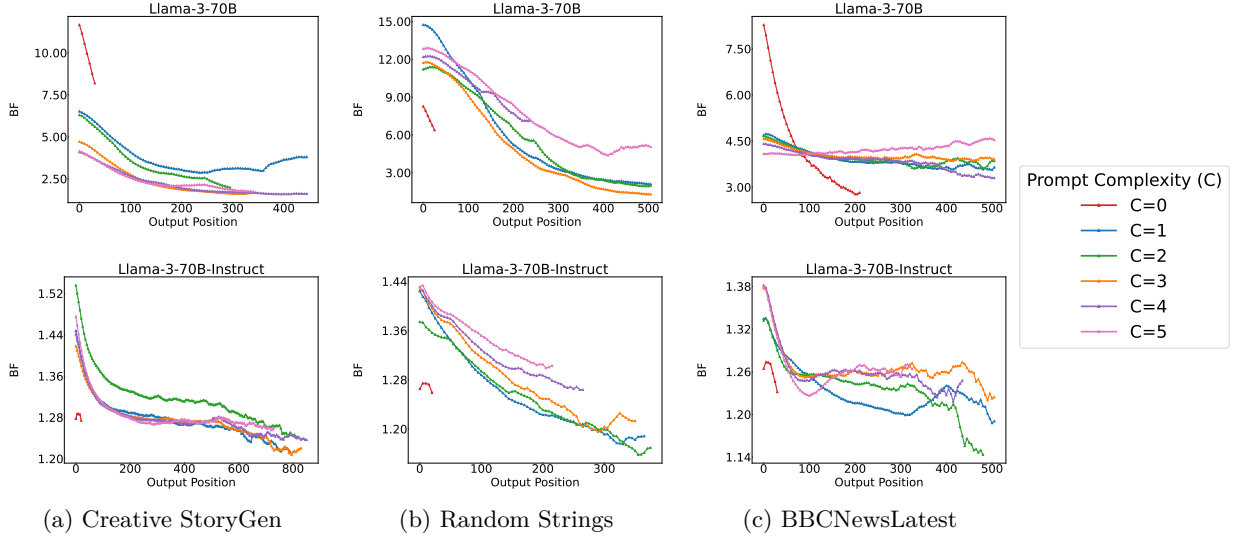


Figure 3: **Shrinking BF with output length over various tasks for Llama-3-70B and Llama-3-70B-Instruct.** For better visualization, we compute the exponential moving averaged values of BF with the smoothing factor set as 0.1.

Impact Factors (IFs). We consider modulating these factors that may impact BF computations: PROMPT COMPLEXITY (C), ALIGNMENT TUNING ($AT \in \{\text{Instruct}, \text{Base}\}$), MODEL SIZE ($S \in \{8\text{B}/13\text{B}, 70\text{B}\}$), and MODEL GENERATION ($G \in \{\text{Llama-2}, \text{Llama-3}\}$). C controls the informativeness of the input prompt x (e.g., the number of banned words in Cognac, the number of in-context samples in MMLU). Intuitively, providing more information in x should make the model more confident in its outputs, resulting in a lower BF. Dataset-specific setups for C are detailed in § B. AT, S, G represent model-wise variations to explore how different configurations of θ affect $B(X; \theta)$.

4.1 BF Dynamic in Generation Process

Both BF and the output length N are functions of the output Y , and BF computation relies on N . To avoid confounding effects, we first analyze how BF varies with N before intervening IFs. In Figure 3, we demonstrate BF trajectories over different output positions by running Llama-3-70B and Llama-3-70B-Instruct on three representative tasks. Specifically, we compute BF over every five output tokens, conditioning on the prompt and all previously generated output tokens.⁷ Our findings also generalize to summarization, multilingual tasks, OLMo-2 (OLMo et al., 2024) /Qwen family (Qwen, 2025) (§ G).

As we can see, first, **the base model’s BF is often significantly higher than the aligned models, roughly 2–5 times.** The nearly order-of-magnitude difference is also a frequent pattern in strongly aligned models when we compare base and aligned models at the beginning of outputs. Therefore, there are actually very few candidate next-token to be truncated in decoding for the aligned models. This explains why the decoding methods exert weaker effects for aligned models (Song et al., 2024; Renze & Guven, 2024; Shi et al., 2024a), as we will see in § 5.1. Also, in most cases, **BF would often drop smoothly as more output tokens are generated.** Under the same task, when $C > 0$, different C mainly controls the starting point and the rate of decreasing, while in the end, they would converge to roughly the same point. When almost zero knowledge is provided ($C = 0$), the output will end much earlier compared to $C > 0$ cases. These findings also provide support that the future token generation is gradually becoming predictable and the model may have a certain generation plan to follow, resonating with recent observations in interpretability (Pal et al., 2023; Wu et al., 2024; Li et al., 2024a) and inference acceleration (Cai et al., 2024; Welleck et al., 2024).

We further examine potential confounds such as prompt likelihood and data contamination in § L, and find they do not fully account for the observed BF reductions.

⁷See § D for full results across all models and tasks.